

Check Palindrome

Check Palindrome

→
r a c e c a r
←

Given a string represented as a character arrays, check it is a **palindrome** or not.

Example

Input : "racecar"

Output : true

Input : "rotator"

Output : true

→
a b c d c b a
←

→
w o w
←

→
n i t i n
←

→
a l o l a
←

→
n o m o n
←

str.len = n

1. create a copy of the given str ← n
2. reverse the copy ← n/2
3. compare the rev. copy with org. str ← n

time: $\frac{5n}{2} \sim O(n)$

space: $O(n)$

2nd appr

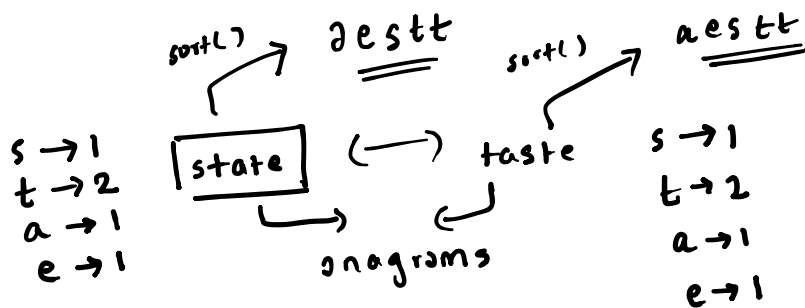
i o i i j j j j
↓ ↓ ↓ ↓ ↓ ↓ ↓ ↓
a b c d c b a
└──────────┘

a b c d e f d c b a
└──────────┘
x

i j
↓ ↓
a b c c b a

		i	l		
a	b	c	c	b	a
↑	↑	↑	↑	↑	↑
o	i	j	i	j	j
i					

Check Anagrams



Check Anagrams

Given a two strings represented as a character arrays, check if they are **anagrams**.

note : assume characters the input strings are lowercase letters (a - z).

Example

Input : "state" and "taste"
Output : true

Input : "abacbac" and "aabbbcc"
Output : false

a b a c b a c
=> a -> 3
=> b -> 2
=> c -> 2

a a b b b c c
=> a -> 2
=> b -> 3
=> c -> 2

sort
a a a b b c c
= a a b b b c c

1. Sort $S_1[]$ and $S_2[]$ $\rightarrow n \log n$
2. cmp mem $\rightarrow n$

time: $2n \log n + n \therefore O(n \log n)$

$S_1 =$ "abacbac"
=> a b a c b a c

=> freq

(a)	(b)	(c)	(d)		(z)
0	1	2	3		25
0	1	2	3		10
1	1	1			
2	2	2			
3					

a -> 0
b -> 1
c -> 2
:
:
z -> 25

ch -> idx
'a' - 'a' = 97 - 97 = 0
'b' - 'a' = 98 - 97 = 1
'c' - 'a' = 99 - 97 = 2
:
:

$$\log_a^b = b$$

appr 1

appr 2 ✓
 $2n + 26$

$$\eta = 2^2$$

$$2n \log n + n$$

$$\begin{aligned} & \underbrace{2 \cdot 2^2} \log_2 \overbrace{2^2} + 2^2 \\ &= 8 \cdot 2 + 4 \\ &= 20 \end{aligned}$$

$$\begin{aligned} \eta = 2^2 \quad & 2 \cdot 2^2 + 2^6 \\ &= 8 + 26 \\ &= 34 \end{aligned}$$

$$\eta = 2^{10}$$

$$\begin{aligned} & \underbrace{2 \cdot 2^{10}} \log_2 \overbrace{2^{10}} + 2^{10} \\ &= 2048 \cdot 10 + 1024 \\ &= 20480 + 1024 \\ &\sim 21K \end{aligned}$$

$$\begin{aligned} & 2 \cdot 2^{10} + 2^6 \\ &= 2 \cdot 1024 + 26 \\ &= 2048 + 26 \\ &\sim 2K \end{aligned}$$

Find Largest of N Strings

Given a **N** strings represented as a character arrays, design an algorithm to find the **lexicographically largest** string.

Example

Input : "abc", "ae", "xyz", "zz"

Output : "zz"

$n = 4$

"abc"

"ae"

"xyz"

"zz"

lexicographically
largest

String Class in C++

type var;

int x;
char h;

dynamic
no need to specify size when creating string obj

```
string str; // declaration of a string object (variable)
```

by default when you create a string obj, it contains empty string

str is a var. of string type
str is an obj of string class

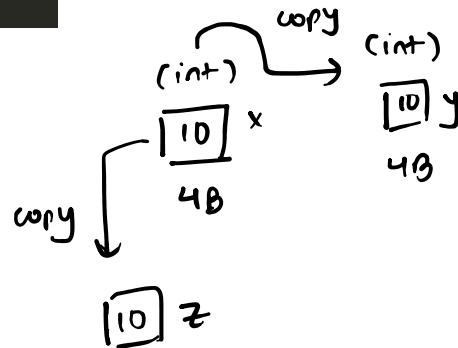
Initialization of a String

int x = 10;

```
string str = "coding blocks";  
cout << str;
```

'\0' is added automatically

int x = 10;
int y = x;
int z;
z = x;



Indexing a String

string s = "coding";

string s = "coding";
 n=6

s[0]

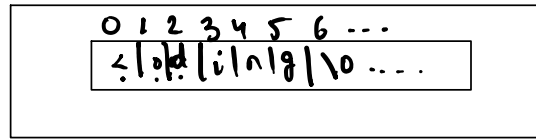
s[1]

s[2]

⋮

s[i]

$0 \leq i \leq n-1$
 ↑
 s.size()



s

Reading Input into a String

```
string str; // object declaration
cin >> str;
```

cin >> stops reading i/p as soon as you encounter a non-leading white-space

```
string str; // object declaration
getline(cin, str);
```

it stop reading i/p as soon as it encounters '\n'

getline(cin, str, '\n');

default delimiting char.

String Concatenation

```

string s1 = "abc";
string s2 = "def";
s1.append(s2);
cout << s1;    "abcdef"
cout << s2;    "def"

```

'+' is defined for string

```
s1 = s1 + s2;
```

String Comparison

```

if (s1 == s2) {
    s1 = "abc"
    s2 = "abc"
    s1.compare(s2) → 0
} else if (s1 > s2) {
    s1 = "abc"
    s2 = "adc"
    s1.compare(s2) → -ve
} else {
    s1 = "abc"
    s2 = "abc"
    s1.compare(s2) → +ve
}

```

>
 >=
 <
 <=
 ==
 !=

all these ops. are defined for str. type

Searching in a String

↓
 "0 1 2 3 4 5 6 7"
 "0 1 2 3 4 5 6 7"

Searching in a String

$\downarrow$
0 1 2 3 4 5 6 7
s = " a b c a b c a b "

"ca"

s.find("ca") $\rightarrow$ 2

s.rfind("ca") $\rightarrow$ 5

Extracting a Substring

string s = " c o d i n g ";
 $\downarrow$
0 1 2 3 4 5
 2
string ss = s.substr(3, 2);
 $\uparrow$
cout << ss; "in"
idx 3 onwards
extract a substring
of len 2

Reversing a String

begin end
 $\downarrow$ $\downarrow$
s = " a b c d e ";

reverse (s.begin(), s.end())
 [first, last)

Sorting a String

" x z a b d "
 $\downarrow$ sort (s.begin(), s.end())

" abdxz "

Sorting a String Sequence

Generate Sub-Strings I

→ for a str. of size n , it'll have $\frac{n(n+1)}{2} \sim n^2$ substrings

Generate Sub-Strings I

Given a string, design an algorithm to generate all of its **sub-strings**.

We define a **sub-string** of an string as a **contiguous** part of the given string.

Example

Input : "abcde"

Output : "a", "ab", "abc", "abcd", "abcde"
"b", "bc", "bcd", "bcde"
"c", "cd", "cde"
"d", "de"
"e"

a b c d e

0 1 2 3 4
a b c d e

$$0 \leq i \leq n-1$$

$$i \leq j \leq n-1$$

i j
j-i+1

s.substr(i, j-i+1)

s

i j



Generate Sub-Strings II

Given a string, design an algorithm to generate all of its **sub-strings** length-wise.

We define a **sub-string** of an string as a **contiguous** part of the given string.

Example

Input : "abcde"

Output : "a", "b", "c", "d", "e"
"ab", "bc", "cd", "de"
"abc", "bcd", "cde"
"abcd", "bcde"
"abcde"

$l=1$	$l=2$	$l=3$	$l=4$	$l=5$
a	ab	abc	abcd	abcde
b	bc	bcd	bcd	
c	cd	cde		
d	de			
e				

Check Good Strings

Check Good Strings

Given a string **str**, design an algorithm to check it is a **good** string.

We define a good string as a string only contains **vowels** [a, e, i, o, u].

Example

Input : "uoiea"
Output : True

==

u o x i a
↑
false

→ assume i/p
contains
just lowercase
letters

Longest Good Substring

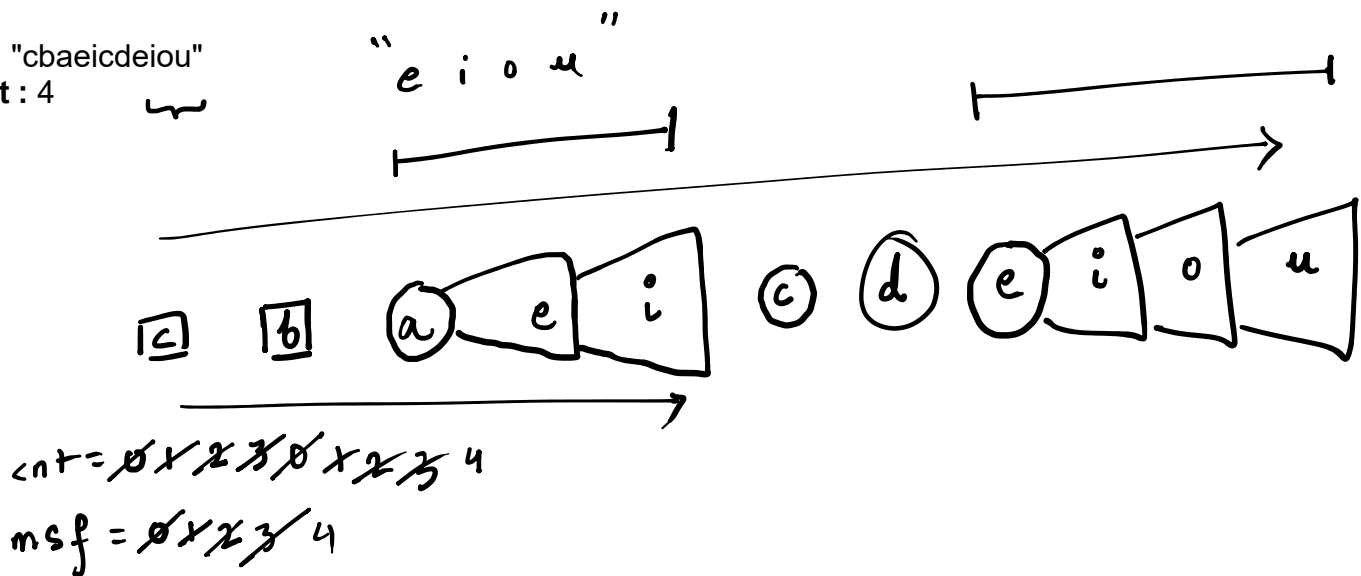
Given a string **str**, design an algorithm to find the length of its **longest good sub-string**.

We define a good sub-string of **str** as a sub-string that only contains **vowels** [a, e, i, o, u].

Example

Input : "cbaeicdeiou"

Output : 4



Rudeness of a String

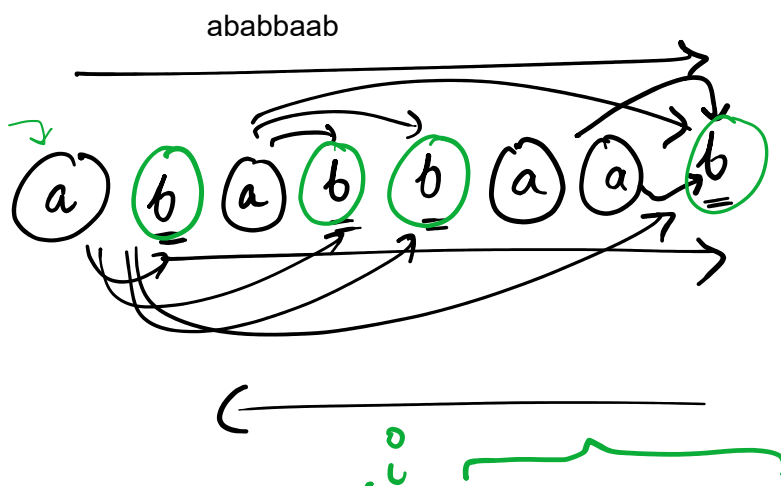
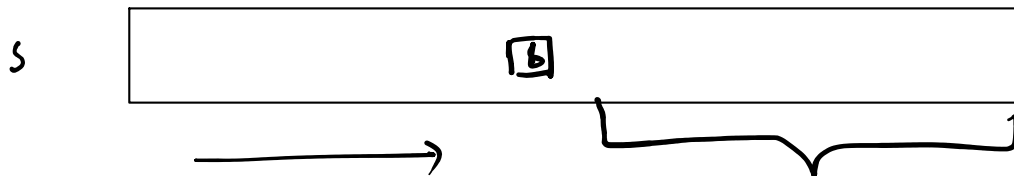
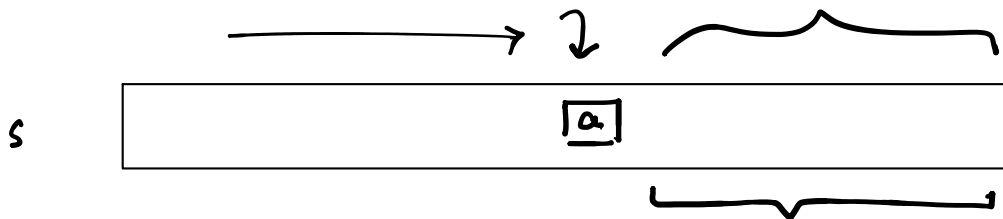
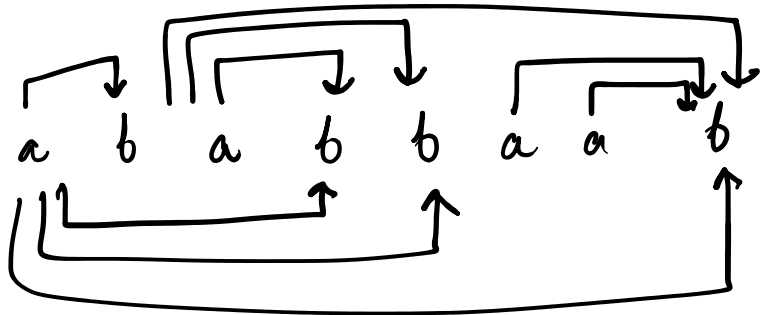
Given a string s containing n lowercase English letters, design an algorithm to find its **rudeness** – we define **rudeness** of a string as the no. of pairs of integers (i, j) in s such that $0 \leq i < j \leq n-1$ and $s_i = 'a'$ and $s_j = 'b'$.

Example

Input : ababbaab

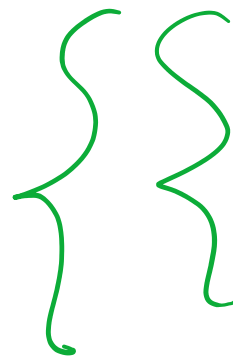
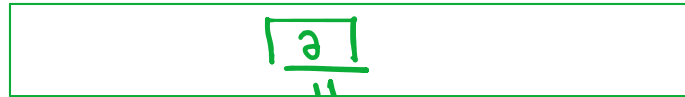
Output : 9

=

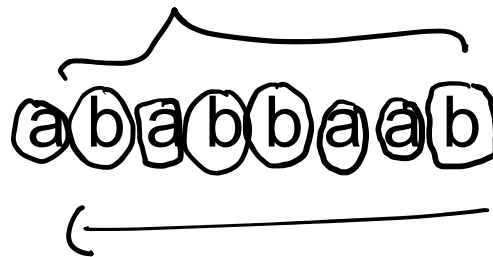


~~rudeness = 0/47~~
89
=

s[i]



find out no. of b's
in $s[p+1 \dots n-1]$
in $O(1)$ time



rudeness = ~~0~~ ~~1~~ ~~2~~ ~~3~~ 4
cnt+b = ~~0~~ ~~1~~ ~~2~~ ~~3~~ 4
=

Count Palindromic Substrings

Given a string **s**, design an algorithm to count the no. of **palindromic substrings** in it.

Example

Input

0	1	2	3	4	5
a	b	a	b	a	a

Output : 11

odd-len

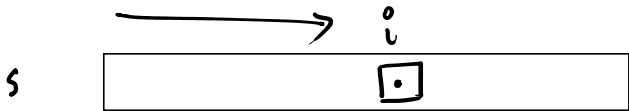
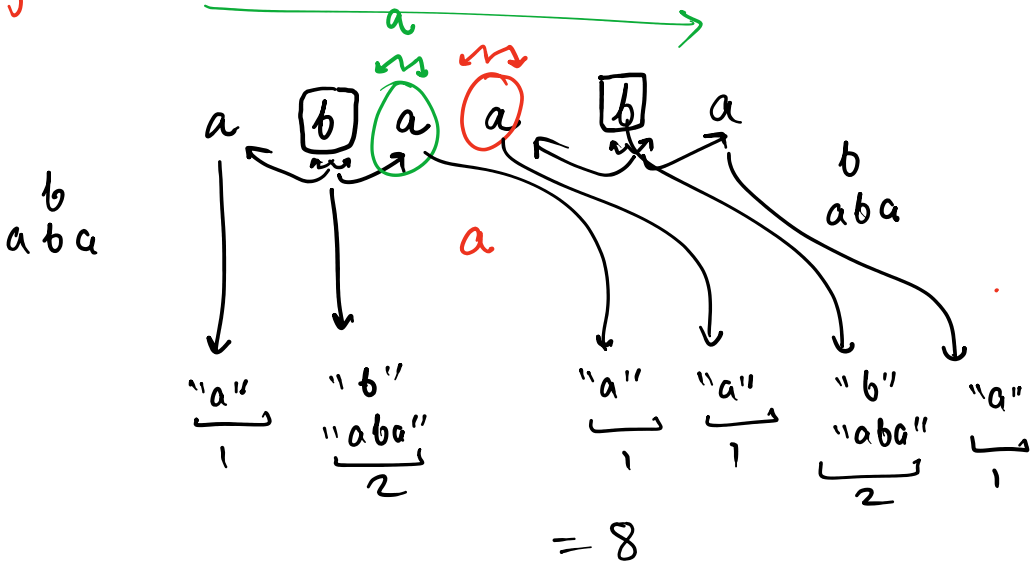
even-len

3

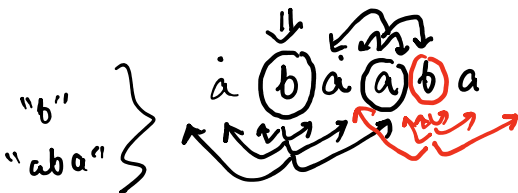
8

11

1. find # odd-len palindromic substrings



find out # odd-len palindromic substrings with $s[i]$ as the center?



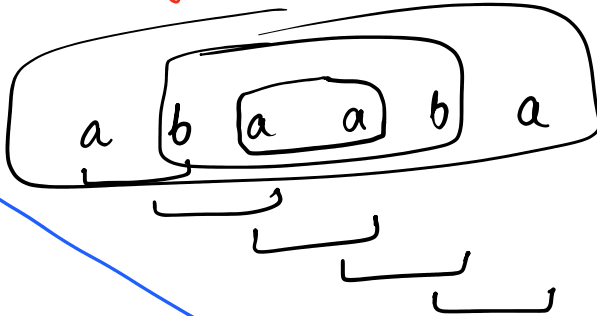
$i-j \geq 0$
and
 $i+j \leq n-1$
and

"aba"

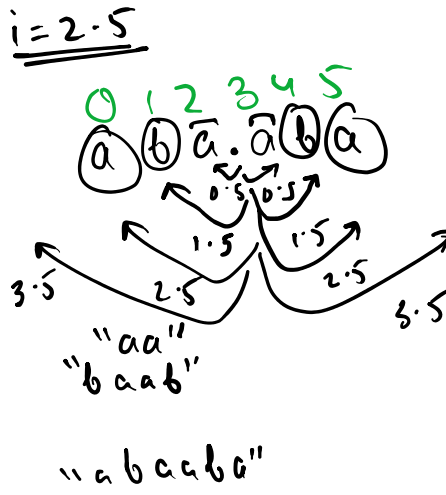
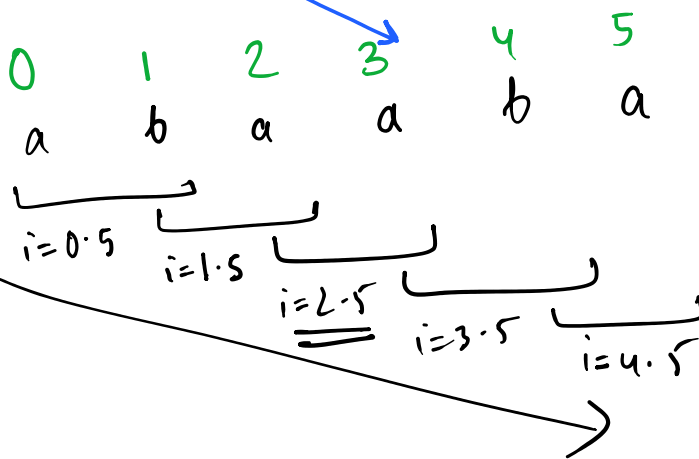


end
 $s[i-j] == s[i+j]$

2. find out no. of even-len palindromic substrings



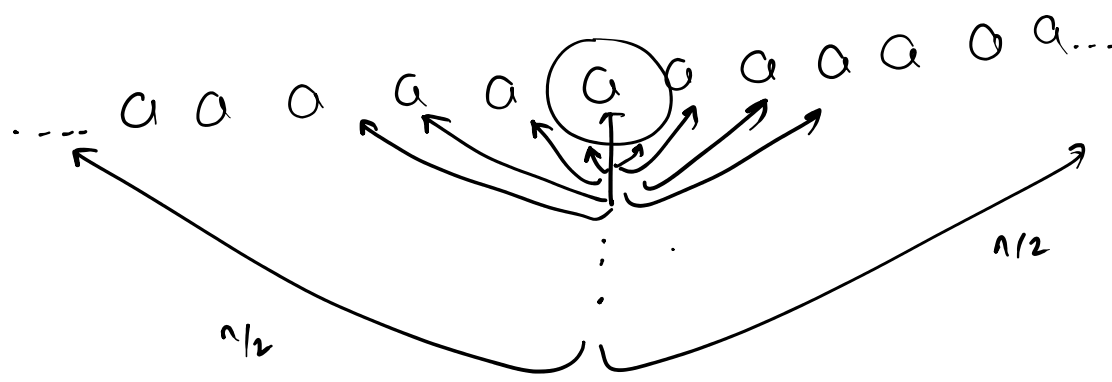
aa
 baab
 abaaaba



#odd-centers = n

#even-centers = $n-1$

$$\frac{2n-1}{2}$$



$$= (2n-1) \cdot \frac{n}{2}$$

$$= \frac{2n^2 - n}{2} \quad \boxed{\sim O(n^2)}$$

Form Biggest Number

Given an integer array, design an algorithm to **arrange** the elements of the array in a way that the arrangement yield the **biggest** number.

Example

Input

0	1	2	3
54	546	548	60

Output : 6054854654

Input

0	1	2	3
98	9	78	7

Output : 998787

$\Rightarrow \{ 60, 548, 546, 54 \}$

6054854654

$\{ 9, 98, 78, 7 \}$

998787